

مبانی و الگوریتم‌های کلان‌داده

جزوه جامع درسی و مرجع دانشگاهی تحلیل کلان‌داده

بر اساس سرفصل‌ها و اسلایدهای کتاب (MMDS) Datasets Massive of Mining
تألیف: ژوره لسکوک، آناند راجارامان، جفری اولمن

گردآورنده: احسان شهبازی

مجموعه یادداشتهای آموزشی کارشناسی ارشد

۷ مهر ۱۴۰۵

فصل ۴

کاوش جریان‌های داده (Data Streams)

۱.۴ مقدمه و مدل محاسباتی جریان داده

در سیستم‌های سنتی پایگاه داده، فرض بر این است که داده‌ها ابتدا روی دیسک به صورت پایدار ذخیره می‌شوند و سپس پرس‌وجوهای تحلیلی روی داده‌های ایستا اجرا می‌گردند. اما در بسیاری از کاربردهای نوین کلان‌داده، داده‌ها به صورت پیوسته، با سرعتی سرسام‌آور و در حجم نامحدود تولید می‌شوند؛ به این نوع داده‌ها **جریان داده**^۱ گفته می‌شود. نمونه‌های بارز جریان داده عبارت‌اند از:

- کلیک‌های کاربران در وبسایت‌های پرترافیک مانند گوگل یا آمازون.
- داده‌های ارسالی از میلیون‌ها حسگر در شبکه‌های اینترنت اشیا (IoT).
- بسته‌های اطلاعاتی عبوری از روترهای شبکه اینترنت (نظیر پایش حملات DDoS و ناهنجاری‌های شبکه).
- پیام‌های منتشرشده در شبکه‌های اجتماعی نظیر توییتر.

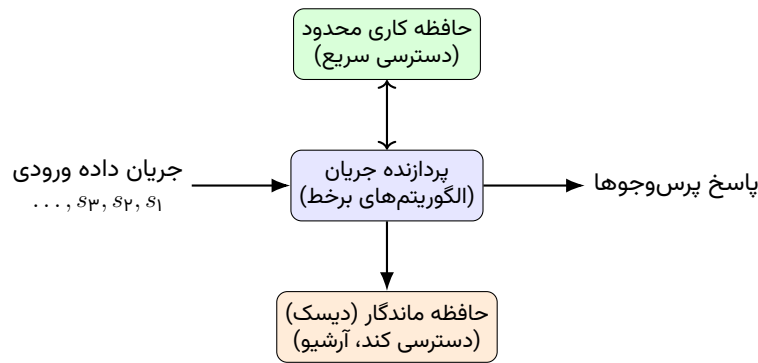
تعریف ۱.۴: مدل پردازش جریان داده

در مدل محاسباتی جریان داده (شکل ۱.۴):

۱. داده‌ها به شکل دنباله‌ای بی‌پایان از تاپل‌ها وارد سیستم می‌شوند.
۲. نرخ ورود داده‌ها ممکن است بسیار فراتر از توان ذخیره‌سازی فوری روی دیسک باشد.
۳. سیستم پردازش جریان دارای یک **حافظه کاری**^a با ظرفیت محدود است.
۴. هر عنصر تنها یک بار (یا با تعداد دفعات بسیار محدود) قابل مشاهده است؛ اگر عنصری پردازش نشود، برای همیشه از دست می‌رود.

Working Memory^a

Data Stream^۱



شکل ۱.۴: معماری سیستم مدیریت جریان داده: پردازش فوری با تکیه بر حافظه کاری محدود

پرس‌وجوها در جریان داده به دو دسته تقسیم می‌شوند:

- **پرس‌وجوهای دائمی^۲:** پرس‌وجوهایی که همواره فعال هستند و به ازای هر عنصر ورودی، پاسخی به روز تولید می‌کنند (مثلاً: «اگر دمای هر حسگر از ۸۰ درجه بیشتر شد، هشدار بده»).
- **پرس‌وجوهای موردی^۳:** پرس‌وجوهایی که در یک لحظه خاص توسط تحلیل‌گر مطرح می‌شوند (مثلاً: «میانگین طول پیام‌های ارسال‌شده در ۱۰ دقیقه گذشته چقدر بوده است؟»).

۲.۴ نمونه‌گیری از جریان‌های داده

هنگامی که نمی‌توان تمام جریان را ذخیره کرد، نمونه‌گیری یک راهکار طبیعی است. دو مسئله اساسی در نمونه‌گیری وجود دارد:

۱.۲.۴ نمونه‌گیری با نسبت ثابت (Fixed Proportion Sampling)

فرض کنید می‌خواهیم ۱۰٪ از پرس‌وجوهای یک موتور جستجو را نمونه‌برداری کنیم تا رفتار کاربران را بررسی نماییم. اگر به ازای هر پرس‌وجو یک عدد تصادفی بین ۰ تا ۹ تولید کنیم و در صورت صفر بودن آن را برگزینیم، هر کاربر دارای تعداد پرس‌وجوهای متفاوتی در نمونه خواهد بود، اما اگر بخواهیم بفهمیم چه کسری از کاربران تنها یک بار پرس‌وجو کرده‌اند، این نمونه‌گیری نتایج را به شدت مخدوش می‌کند.

راهکار اصولی: نمونه‌گیری بر مبنای **کلید کاربر**. با استفاده از یک تابع درهم‌سازی:

$$h(\text{UserID}) \pmod{10} = 0 \quad (1.4)$$

اگر باقیمانده صفر شد، تمام پرس‌وجوهای آن کاربر ذخیره می‌شود و در غیر این صورت هیچ‌یک از پرس‌وجوهای او ذخیره نمی‌گردد. این کار نسبت نمونه ۱۰٪ را تضمین کرده و در عین حال ساختار کامل رفتار هر کاربر نمونه را حفظ می‌کند.

۲.۲.۴ نمونه‌گیری با اندازه ثابت: الگوریتم مخزنی (Reservoir Sampling)

فرض کنید می‌خواهیم در هر لحظه نمونه‌ای تصادفی و یکنواخت به اندازه دقیقاً s از میان n عنصر نخست جریان که تاکنون مشاهده شده‌اند در اختیار داشته باشیم، در حالی که مقدار نهایی n از پیش معلوم نیست و حافظه ما فقط گنجایش s عنصر را دارد.

الگوریتم ۱.۴: الگوریتم نمونه‌گیری مخزنی (Reservoir Sampling)

- **ورودی:** جریان داده $x_1, x_2, \dots$ و ظرفیت مخزن s .
- **گام اول:** s عنصر نخست جریان $(x_1, \dots, x_s)$ را مستقیماً در مخزن قرار دهید.
- **گام دوم:** برای هر عنصر ورودی بعدی x_n (که $n > s$):

۱. با احتمال $\frac{s}{n}$ عنصر x_n را برای ورود به مخزن بپذیرید.
۲. در صورت پذیرش، یکی از s عنصر موجود در مخزن را به صورت کاملاً تصادفی و یکنواخت (با احتمال $1/s$) انتخاب کرده و عنصر جدید را جایگزین آن نمایید.
۳. با احتمال $1 - \frac{s}{n}$ عنصر جدید را بدون تغییر مخزن دور بریزید.

قضیه ۱.۴: صحت الگوریتم مخزنی

پس از رسیدن n عنصر از جریان، احتمال حضور هر عنصر x_i ($1 \leq i \leq n$) در مخزن دقیقاً برابر با $\frac{s}{n}$ است.

اثبات با استقرای ریاضی:

- **پایه استقرا:** برای $n = s$ ، تمامی s عنصر اول در مخزن قرار دارند و احتمال حضور هر یک $s/s = 1$ است که درست است.
- **فرض استقرا:** فرض کنیم حکم برای $n - 1$ عنصر برقرار باشد؛ یعنی بعد از $n - 1$ عنصر، هر عنصری با احتمال $\frac{s}{n-1}$ در مخزن قرار دارد.
- **گام استقرا:** حال عنصر n -ام می‌رسد:

۱. برای خود عنصر n -ام، الگوریتم مستقیماً آن را با احتمال $\frac{s}{n}$ در مخزن قرار می‌دهد.
۲. برای هر عنصر قبلی x_i ($i < n$): این عنصر در مخزن باقی می‌ماند اگر و تنها اگر قبل از ورود عنصر n -ام در مخزن بوده باشد، و با ورود عنصر جدید حذف نشود:

$$\begin{aligned}
 \Pr[x_i \text{ حذف نشود} \mid n] &= \Pr[x_i \text{ در مخزن در گام } n-1] \times \Pr[x_i \text{ حذف نشود} \mid n] \\
 &= \left(\frac{s}{n-1}\right) \times \left(1 - \frac{s}{n} + \frac{s}{n} \left(1 - \frac{1}{s}\right)\right) \\
 &= \left(\frac{s}{n-1}\right) \times \left(1 - \frac{s}{n} + \frac{s}{n} - \frac{1}{n}\right) \\
 &= \left(\frac{s}{n-1}\right) \times \left(\frac{n-1}{n}\right) = \frac{s}{n}
 \end{aligned} \tag{۲.۴}$$

بنابراین حکم برای تمامی n عنصر اثبات می‌شود.

۳.۴ فیلتر کردن جریان‌های داده: فیلتر بلوم (Bloom Filter)

فرض کنید مجموعه‌ای از m کلید مجاز یا نامطلوب S داریم (مثلاً ۱ میلیارد آدرس ایمیل اسپم یا های URL فیشینگ). جریان عظیمی از تاپل‌ها وارد می‌شود و می‌خواهیم در چند میکروثانیه بررسی کنیم آیا کلید ورودی در S هست یا خیر. اندازه S ممکن است به گیگابایت‌ها برسد و در حافظه کش سرور جا نگیرد.

تعریف ۲.۴: فیلتر بلوم [۱]

- یک **فیلتر بلوم**^a یک ساختار داده احتمالی و بسیار فشرده برای آزمون عضویت در مجموعه است که:
- دارای یک آرایه بیتی B به طول n (در ابتدا همگی ۰) است.
 - از k تابع درهم‌سازی مستقل $h_1, h_2, \dots, h_k$ استفاده می‌کند که بازه خروجی آنها $[0, n-1]$ است.
 - **درج عضو** $x \in S$: برای تمام $i \in \{1, \dots, k\}$ بیت $B[h_i(x)]$ برابر با ۱ قرار داده می‌شود.
 - **پرس‌وجوی عضویت** y : اگر برای تمامی $i \in \{1, \dots, k\}$ مقدار $B[h_i(y)] = 1$ باشد، پاسخ «بله» است؛ وگرنه «خیر».

^aBloom Filter

ویژگی حیاتی فیلتر بلوم:

- **عدم وجود منفی کاذب (Zero False Negatives):** اگر عنصری در S باشد، بیت‌های مربوط به آن قطعاً ۱ شده‌اند و امکان ندارد پاسخ منفی دریافت کند.
- **احتمال مثبت کاذب (False Positives):** ممکن است عنصری در S نباشد، اما تصادفاً تمامی بیت‌های متناظر با توابع هش آن توسط سایر عناصر ۱ شده باشند.

۱.۳.۴ تحلیل احتمال خطای مثبت کاذب و مقدار بهینه k

فرض کنید m عنصر در فیلتر با اندازه n بیت و با k تابع هش درج شده باشند:

۱. احتمال اینکه یک بیت خاص پس از یک بار اعمال هش همچنان ۰ باقی بماند برابر است با $1 - \frac{1}{n}$.
۲. پس از درج m عنصر با k تابع هش (مجموعاً $k \cdot m$ درج بیت)، احتمال اینکه یک بیت هنوز ۰ باشد:

$$p_0 = \left(1 - \frac{1}{n}\right)^{km} \approx e^{-km/n} \quad (۳.۴)$$

۳. بنابراین، کسر بیت‌هایی از آرایه که مقدار ۱ به خود گرفته‌اند:

$$p_1 = 1 - e^{-km/n} \quad (۴.۴)$$

۴. احتمال اینکه برای یک عنصر ناموجود y ، تمامی k بیت هش‌شده برابر ۱ باشند (نرخ مثبت کاذب):

$$P(\text{Positive False}) = (p_1)^k = \left(1 - e^{-km/n}\right)^k \quad (۵.۴)$$

با مشتق‌گیری نسبت به k و مساوی صفر قرار دادن آن، **تعداد بهینه توابع درهم‌سازی** برای کمینه‌سازی خطای مثبت کاذب به دست می‌آید:

$$k_{\text{opt}} = \frac{n}{m} \ln 2 \approx 0.693 \times \frac{n}{m} \quad (۶.۴)$$

و با جایگذاری این مقدار، کمترین احتمال مثبت کاذب ممکن برابر است با:

$$P_{\text{opt}} = \left(\frac{1}{2}\right)^k = (0.6185)^{n/m} \quad (۷.۴)$$

مثال کاربردی ۱.۴: محاسبه پارامترهای فیلتر بلوم

فرض کنید $m = 10^9$ کلید داریم و می‌خواهیم نرخ مثبت کاذب کمتر از ۱٪ (۰/۰۱) باشد.

$$\bullet \text{ از رابطه } k \approx 7 \Rightarrow 2^k \geq 100 \Rightarrow P \approx (0.5)^k \leq 0.01$$

$$\bullet \text{ از رابطه } \frac{n}{m} \approx 10.7 \Rightarrow k = \frac{n}{m} \ln 2 \Rightarrow \frac{n}{m} = \frac{7}{\ln 2}$$

• بنابراین با اختصاص حدود ۱۰ بیت (تنها ۲۵.۱ بایت) به ازای هر کلید، خطای مثبت کاذب به زیر ۱ درصد می‌رسد!

۴.۴ شمارش عناصر متمایز: الگوریتم فلژوله-مارتین (Flajolet-Martin)

مسئله: در یک جریان از شناسه‌ها، چه تعداد عنصر یکتا و متمایز وجود دارد؟ اگر بخواهیم شناسه‌ها را در جدول درهم‌سازی ذخیره کنیم، با افزایش تنوع عناصر با کمبود شدید حافظه مواجه می‌شویم. الگوریتم فلژوله-مارتین (FM) [۷] تنها با مصرف حافظه‌ای در حد چند کلمه، این مقدار را تخمین می‌زند.

الگوریتم ۲.۴: الگوریتم فلزوله-مارتین برای شمارش عناصر متمایز

• **تابع هش:** تابعی مانند $h(a)$ که عناصر را به رشته‌ای از بیت‌ها به طول L نگاشت می‌کند (به طوری که $2^L > N$).

• **تابع صفرهای انتهایی $r(a)$:** تعداد صفرهای متوالی در سمت راست نمایش دودویی $h(a)$. (مثلاً اگر $h(a) = 101000$ باشد، $r(a) = 3$ است).

• **نگهداری بیشینه صفرهای انتهایی:**

$$R = \max_{a \in \text{Stream}} r(a) \quad (۸.۴)$$

• **تخمین تعداد عناصر متمایز:**

$$\hat{D} = 2^R / \phi \quad \text{که در آن } \phi \approx 0.77351 \quad (۹.۴)$$

بینش شهودی و تحلیل آماری: احتمال اینکه هش یک عنصر با حداقل i صفر متوالی تمام شود برابر با 2^{-i} است. اگر D عنصر متمایز در جریان وجود داشته باشد، احتمال اینکه هیچ‌کدام از آن‌ها با i صفر خاتمه نیابند:

$$(1 - 2^{-i})^D \approx e^{-D \cdot 2^{-i}} \quad (۱۰.۴)$$

• اگر $2^i \ll D$ باشد، احتمال مشاهده نکردن صفر انتهایی به سمت صفر میل می‌کند (قطعاً چنین عنصری دیده می‌شود).

• اگر $2^i \gg D$ باشد، این احتمال نزدیک به ۱ است (تقریباً محال است چنین عنصری دیده شود).

• نقطه گذار دقیقاً در $2^R \approx D$ اتفاق می‌افتد.

۱.۴.۴ تکنیک کاهش واریانس: میانگین میانه‌ها (Median of Means)

چون 2^R همواره توانی از دو است، واریانس این تخمین‌گر بالاست. برای رفع این نقیصه:

۱. از چندین تابع هش مستقل استفاده می‌کنیم.
۲. توابع هش را به گروه‌های کوچکی افراز می‌کنیم.
۳. در هر گروه، میانگین حسابی^۴ مقادیر 2^R را محاسبه می‌کنیم تا واریانس کاهش یابد.
۴. میان گروه‌ها، میانه^۵ مقادیر میانگین را برمی‌گزینیم تا مقادیر پرت و نامتعارف بی‌اثر شوند.

۵.۴ شمارش بیت‌ها در پنجره لغزان: الگوریتم DGIM

فرض کنید جریانی از صفرها و یک‌ها داریم و می‌خواهیم در هر لحظه تعداد بیت‌های ۱ موجود در پنجره لغزان^۶ به طول N گذشته را بدانیم. ذخیره کل N بیت نیازمند N بیت حافظه است که برای پنجره‌های بزرگ (مثلاً $N = 10^9$) بسیار پرهزینه است. الگوریتم DGIM (ابداع‌شده توسط داتار، گیونیس، ایندیک و موتوانی [۵]) این مسئله را در فضای $O(\log^2 N)$ و با خطای حداکثر ۵۰٪ حل می‌کند.

تعریف ۳.۴: ساختار سطل‌های DGIM

الگوریتم پنجره را به صورت مجموعه‌ای از سطل‌ها^۸ خلاصه می‌کند:

۱. هر سطل شامل دو مقدار است: (۱) زمان انتهایی سطل (یک برچسب زمانی $O(\log N)$ بیتی)؛ (۲) اندازه سطل

Average^۴Median^۵Sliding Window^۶

(تعداد بیت‌های ۱ موجود در آن).

۲. اندازه هر سطل همواره توانی از ۲ است $(1, 2, 4, 8, \dots)$.

۳. قاعده ظرفیت: به ازای هر اندازه سطل، حداکثر دو سطل (یک یا دو سطل) در پنجره مجاز است و هرگز سه سطل با اندازه یکسان نباید وجود داشته باشد.

۴. سطل‌ها هرگز هم‌پوشانی ندارند و سطل‌های با اندازه بزرگ‌تر، برچسب‌های زمانی قدیمی‌تری دارند.

Buckets^a

الگوریتم ۳.۴: به‌روزرسانی و پاسخ به پرس‌وجو در الگوریتم DGIM

عملیات به‌روزرسانی هنگام ورود بیت جدید:

۱. اگر بیت جدید ۰ باشد، کاری انجام نمی‌شود (تنها برچسب زمان جاری پیش می‌رود).

۲. اگر بیت جدید ۱ باشد:

- سطلی با اندازه ۱ و با زمان جاری ایجاد می‌شود.
- اگر اکنون سه سطل با اندازه ۱ داشته باشیم، دو سطل قدیمی‌تر در هم ادغام شده و یک سطل با اندازه ۲ تشکیل می‌دهند.
- این ادغام به صورت زنجیره‌ای به سمت اندازه‌های بزرگ‌تر ادامه می‌یابد تا قانون «حداکثر دو سطل از هر اندازه» برقرار بماند.

۳. هر سطلی که زمان انتهای آن قدیمی‌تر از N واحد زمانی گذشته باشد، از انتهای پنجره حذف می‌گردد.

پاسخ به پرس‌وجوی «تعداد اها در k موقعیت اخیر» $(k \leq N)$:

- تمامی سطل‌هایی را که کاملاً در بازه k قرار دارند پیدا کرده و اندازه‌های آن‌ها را به طور کامل جمع بزنید.
- برای آخرین سطل که تا حدی با ابتدای پنجره تداخل دارد، دقیقاً نیمی از اندازه آن را به حاصل جمع اضافه کنید.

قضیه ۲.۴: کران خطای الگوریتم DGIM

حداکثر درصد خطای الگوریتم DGIM هرگز از ۵۰٪ تجاوز نمی‌کند.

اثبات: فرض کنید آخرین سطل با اندازه 2^j دارای تداخل باشد. تخمین ما از این سطل برابر با 2^{j-1} است؛ لذا حداکثر خطای مطلق ممکن برابر با $2^{j-1} - 1$ است. از طرفی، چون قبل از این سطل حداقل یک سطل از هر یک از اندازه‌های $1, 2, 4, \dots, 2^{j-1}$ وجود دارد، مجموع واقعی اها حداقل برابر با $2^j = \sum_{i=0}^{j-1} 2^i + 1$ خواهد بود. بنابراین نسبت خطا:

$$\text{Error} \leq \frac{2^{j-1}}{2^j} = 50\% \quad (11.4)$$

(با افزایش سقف تعداد سطل‌ها از ۲ به r ، می‌توان خطا را به دلخواه تا حد $1/r$ کاهش داد).

۶.۴ تخمین گشتاورهای جریان: الگوریتم آلون-ماتیاس-سگدی (AMS)

در تحلیل آماری جریان، گشتاور k -ام^v به صورت زیر تعریف می‌شود:

$$F_k = \sum_{v \in V} m_v^k \quad (12.4)$$

^v k -th Moment

که در آن m_v فراوانی رخداد مقدار v در جریان است:

• **گشتاور صفرم** (F_0): تعداد عناصر متمایز است (توسط الگوریتم فلژوله-مارتین حل شد).

• **گشتاور یکم** (F_1): طول کل جریان ($N = \sum m_v$).

• **گشتاور دوم** (F_2): **عدد شگفتی** (Surprise Number) نام دارد و مقیاسی از میزان کجی (چولگی) یا ناهمگونی توزیع است. اگر همه عناصر فراوانی یکسان داشته باشند، F_2 کمینه است؛ و اگر یک عنصر کل جریان را تسخیر کند، F_2 بیشینه (N^2) خواهد بود.

الگوریتم ۴.۴: AMS برای محاسبه گشتاور دوم F_2

الگوریتم Alon-Matias-Szegedy یک برآوردگر نارایب شگفت‌انگیز برای F_2 در فضای محدود است:

۱. یک موقعیت تصادفی p در بازه زمانی $[1, N]$ انتخاب کرده و عنصر دیده‌شده در آن لحظه (v) را ذخیره کنید.

۲. متغیری به نام c تعریف کرده و تعداد دفعات مشاهده مجدد v را از موقعیت p تا انتهای جریان بشمارید (در ابتدا $c = 1$).

۳. در پایان، مقدار برآوردگر را به صورت زیر محاسبه نمایید:

$$X = N(2c - 1) \quad (13.4)$$

قضیه ۳.۴: نارایب بودن برآوردگر AMS

امید ریاضی متغیر تصادفی X در الگوریتم AMS دقیقاً برابر با گشتاور دوم واقعی F_2 است:

$$\mathbb{E}[X] = F_2 \quad (14.4)$$

اثبات: فرض کنید عنصر v با فراوانی کلی m_v در جریان وجود دارد. احتمال اینکه موقعیت تصادفی p بر روی یکی از m_v وقوع قرار گیرد برابر با $1/N$ است. اگر بر روی i -امین وقوع از آخر قرار گیرد، متغیر شمارش برابر با i خواهد شد ($i \in \{1, \dots, m_v\}$). بنابراین:

$$\mathbb{E}[X] = \sum_{v \in V} \frac{1}{N} \sum_{i=1}^{m_v} N(2i - 1) = \sum_{v \in V} \sum_{i=1}^{m_v} (2i - 1) \quad (15.4)$$

چون مجموع m عدد فرد اول برابر با مربع کامل m^2 است $(\sum_{i=1}^m (2i - 1) = 1 + 3 + 5 + \dots + (2m - 1) = m^2)$

$$\mathbb{E}[X] = \sum_{v \in V} m_v^2 = F_2 \quad (16.4)$$

با نگهداری چندین نمونه متغیر تصادفی X و محاسبه میانگین میانه‌های آن‌ها، واریانس تخمین به سرعت کاهش می‌یابد.

۷.۴ پنجره‌های با کاهش نمایی (Decaying Windows)

در بسیاری از کاربردهای تحلیل جریان (نظیر روندهای توییت‌ر یا معاملات بورس)، رخدادهای اخیر از اهمیت بسیار بالاتری نسبت به رویدادهای دوردست برخوردارند. در **پنجره با کاهش نمایی**^۸، وزن هر رویداد با گذشت زمان به صورت نمایی مستهلک می‌شود:

$$w(t) = e^{-\lambda(t_{\text{now}} - t)} \approx (1 - c)^{t_{\text{now}} - t} \quad (17.4)$$

مزیت بی‌نظیر این رویکرد در محاسبات کلان‌داده این است که برای به‌روزرسانی مجموع اوزان در هنگام ورود یک عنصر جدید، نیازی به نگهداری هیچ تاریخچه‌ای نیست؛ تنها کافی است مقدار مجموع پیشین را در ضریب تخفیف $(1 - c)$ ضرب کرده و عدد ۱ را به آن بیفزاییم:

$$S_{\text{new}} = S_{\text{old}} \times (1 - c) + 1 \quad (18.4)$$

این فرمول در زمان و فضای ثابت $O(1)$ اجرا شده و بار محاسباتی سرورهای تحلیل بلادرنگ را به صفر نزدیک می‌کند.

جمع‌بندی فصل چهارم

- در جریان‌های داده، امکان ذخیره‌سازی کل تاریخچه وجود ندارد؛ پردازش باید یک‌گذره و با حافظه زیرخطی انجام شود.
- نمونه‌گیری مخزنی به انتخاب تصادفی یکنواخت از میان عناصر با اندازه نامعلوم جریان امکان می‌دهد.
- فیلتر بلوم با حداقل مصرف حافظه، عضویت در مجموعه‌ها را با صفر درصد منفی کاذب بررسی می‌کند.
- الگوریتم Flajolet-Martin شمارش عناصر متمایز را از روی بیشینه صفرهای متوالی درهم‌سازی تخمین می‌زند.
- الگوریتم DGIM شمارش عناصر پنجره لغزان را در فضای $O(\log^2 N)$ با خطای تضمین‌شده کنترل می‌نماید.